

Text Processing

Computers need text turned into units and numbers before any model can use it. This module covers the first steps of every NLP pipeline.

Tokenization

Prerequisites: none beyond basic Python strings.

Tokenization splits text into smaller units called tokens. The simplest approach splits on whitespace and punctuation, so "Students can't wait!" might become "Students", "can", "'t", "wait", "!". Even this simple step involves decisions about contractions, hyphens, numbers and emojis.

Word-level tokenization creates huge vocabularies and cannot handle words it has never seen, called out-of-vocabulary (OOV) words. Character-level tokenization has no OOV problem but produces very long sequences.

Modern language models use subword tokenization, which splits rare words into frequent pieces: "tokenization" might become "token" and "ization". Byte Pair Encoding (BPE) builds the vocabulary by repeatedly merging the most frequent pair of adjacent symbols; WordPiece, used by BERT, is a close relative.

Tokens are also the unit of cost for LLM APIs. For English text, one token is roughly four characters, or about three quarters of a word, so 1,000 tokens is about 750 words. Languages other than English, such as Hindi, often need more tokens for the same content.

Stemming and Lemmatization

Prerequisites: Tokenization.

Both techniques reduce words to a base form so that "studies", "studying" and "studied" can be treated as the same word, which shrinks the vocabulary and helps classic keyword search.

Stemming chops off endings using simple rules, without understanding the word. The Porter stemmer, published in 1980, is the classic example. It is fast but crude: "studies" becomes "studi", and "university" and "universe" can end up with the same stem.

Lemmatization uses a dictionary and the word's part of speech to return a real word called the lemma: "studies" becomes "study", and "better" (as an adjective) becomes "good". It is more accurate but slower and needs language resources such as WordNet.

Stop-word removal often happens at the same stage: very common words such as "the", "is" and "of" are dropped because they carry little meaning for keyword-based methods.

Modern neural models generally skip stemming, lemmatization and stop-word removal, because subword tokenization and context-aware embeddings handle word variations themselves. These steps remain useful for keyword search and small classic models.

Bag of Words and TF-IDF

Prerequisites: Tokenization.

Bag of Words (BoW) represents a document as a vector of word counts over a fixed vocabulary. Word order is thrown away, hence the "bag". Two documents with similar counts are treated as similar. The vectors are very long and mostly zeros, which is called sparse.

Raw counts over-reward common words. TF-IDF fixes this by weighting each word by two factors. Term frequency (TF) is how often the word appears in this document. Inverse document frequency (IDF) is the logarithm of the total number of documents divided by the number of documents containing the word. A word that appears in every document gets an IDF near zero; a word that is frequent in one document but rare overall gets a high score, which makes it a good keyword for that document.

TF-IDF is fast, needs no training and works well for keyword search and simple classifiers. Its weakness is that it only matches exact words: "car" and "automobile" share nothing. BM25, a refined relative of TF-IDF, is still the standard keyword ranking function in search engines and is combined with embeddings in hybrid search.